

A Comparative Study of Classical Machine Learning Algorithms for Multi-Class Network Intrusion Detection

Softwarica College of IT & E-Commerce
(Coventry University)



Submitted by
SUDIP ADHIKARI
(250578)

Abstract

This report documents the design, implementation, and evaluation of a comparative study of three classical machine learning algorithms for multi-class network intrusion detection. Traditional, rule-based intrusion detection systems struggle with attacks that do not match a stored signature, and machine learning offers a way to flag suspicious traffic by learning patterns rather than memorising rules. Three algorithms — Logistic Regression, Random Forest, and XGBoost — are trained on the NSL-KDD benchmark dataset to classify traffic into Normal and four attack categories: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). The pipeline performs identical preprocessing (one-hot encoding, standardisation), addresses class imbalance with SMOTE applied per cross-validation fold, and evaluates with stratified 5-fold cross-validation followed by a held-out evaluation on the official KDDTest+ split. XGBoost achieves the best overall test accuracy (78.66%) and the best macro F1 (0.6337), with strong rare-class performance. Logistic Regression remains competitive on weighted F1 thanks to majority-class strength, and Random Forest is the fastest to train. The report also documents an honest finding that many published studies skip: a 22-percentage-point gap between cross-validation and held-out test scores, caused by NSL-KDD deliberately including unseen attack variants in its test split. Feature importance analysis identifies a small set of network flow features (`service_eco_i`, `flag_S0`, `dst_host_serror_rate`) that drive most predictions.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Project Objectives	3
1.3	Why Three Algorithms (Down From Six)	3
1.4	Report Structure	4
2	Dataset: NSL-KDD	4
2.1	Why NSL-KDD	4
2.2	Structure of the Dataset	4
2.3	Class Distribution and the Imbalance Problem	5
2.4	Attack Categories Explained	5
3	Methodology	6
3.1	High-Level Pipeline	6
3.2	Preprocessing	6
3.2.1	Encoding categorical features	6
3.2.2	Standardisation	6
3.3	Class Imbalance: SMOTE	6
3.4	Algorithms	7
3.4.1	Logistic Regression	7
3.4.2	Random Forest	7
3.4.3	XGBoost	7
3.5	Evaluation	7
3.5.1	Cross-validation and held-out test	7
3.5.2	Metrics	7
4	Implementation	8
4.1	Project Layout	8
4.2	Loading and Label Mapping	8
4.3	Preprocessing	8
4.4	Per-Fold SMOTE	9

4.5	Models	9
4.6	Evaluation Utilities	10
4.7	Reproducibility	10
4.8	Source Code Repository	10
5	Results and Discussion	10
5.1	Overall Comparison	10
5.2	The Cross-Validation versus Held-Out Gap	11
5.3	Per-Class Performance	11
5.4	Confusion Matrices	11
5.5	Feature Importance	13
5.6	Wall-Clock Time	14
5.7	Summary of Findings	14
6	Limitations and Future Work	14
7	Conclusion	15
	References	15
	Appendix A: Environment and Reproduction	16
	Appendix B: Selected Code Listings	17
	Appendix C: Full Per-Class Tables	17

1 Introduction

1.1 Background and Motivation

Cyberattacks have become a daily fact of life for organisations of every size. Traditional firewalls and intrusion detection systems (IDS) match incoming network traffic against a database of attack signatures and raise an alert when there is a match. This works well for attacks that have already been catalogued, but the moment an attacker tweaks an exploit or invents a new one, signature-based detection becomes blind. Machine learning offers a complementary route: rather than encoding rules by hand, an ML-based detector learns the difference between benign and malicious traffic from labelled examples and can sometimes generalise to attacks it has not seen before.

The catch is that “machine learning” is not a single thing. Different algorithms make different assumptions about the data, behave differently under class imbalance, and have very different training costs. The natural question — and the question this project answers — is: *when you sit down to build an ML-based intrusion detector, which algorithm should you start with, and what trade-offs should you expect?*

1.2 Project Objectives

The project has four objectives.

1. Build a reproducible pipeline that loads the NSL-KDD benchmark, preprocesses the data, and trains classifiers under identical conditions.
2. Compare three classical ML algorithms — Logistic Regression, Random Forest, and XGBoost — on the full five-class problem, not just the easy binary version.
3. Report *per-class* precision, recall, and F1 in addition to overall accuracy, because accuracy alone is misleading on data this imbalanced.
4. Analyse which network features contribute most to detection so the result is not a black box but something a network analyst could act on.

1.3 Why Three Algorithms (Down From Six)

The original project proposal listed six algorithms (LR, KNN, Decision Tree, Random Forest, SVM, XGBoost). After review, the supervisor suggested narrowing the scope to three so that each algorithm could be analysed properly rather than skimmed. The three were chosen so that each represents a different paradigm:

- **Logistic Regression** — linear, probabilistic, simple. The natural baseline.
- **Random Forest** — a bagging ensemble of decision trees. Robust on tabular data, handles mixed feature types, returns a meaningful feature importance.
- **XGBoost** — a sequential boosting ensemble. The current go-to model for tabular classification.

This trio answers a clean question: *how much detection power do you actually gain by moving from a simple linear model to bagged trees, and from bagged trees to boosted trees?*

1.4 Report Structure

Section 2 describes the NSL-KDD dataset, its class structure, and why it is the standard benchmark in this area. Section 3 walks through the methodology: the preprocessing pipeline, how class imbalance is handled with SMOTE, what each algorithm does in plain terms, and how evaluation is set up. Section 4 details the implementation, including the project layout and the most important code blocks. Section 5 reports the experimental results — overall comparison, per-class performance, confusion matrices, feature importance — and discusses what the numbers mean. Section 6 is honest about what this study does not do, and Section 7 closes with what was learned.

2 Dataset: NSL-KDD

2.1 Why NSL-KDD

NSL-KDD [1] is the most widely used benchmark in classical-ML intrusion detection research. It is a refined version of the original KDD Cup 1999 dataset, with the duplicate records that plagued the original removed. The presence of duplicates in KDD Cup 1999 had biased classifiers toward frequent attack patterns and inflated reported accuracy figures, so NSL-KDD has been the de-facto baseline since 2009.

For this project NSL-KDD has three practical advantages: it is freely available, it ships in well-defined train and test splits so results are directly comparable with previous work, and it has a level of documentation that lets us spend our time on modelling rather than data wrangling. The honest disadvantage is that the underlying network traffic is from 1998 and lacks modern attack patterns. We address this in Section 6 and discuss CICIDS2017 as a follow-up benchmark.

2.2 Structure of the Dataset

NSL-KDD ships in two splits:

- **KDDTrain+** – 125,973 connection records.
- **KDDTest+** – 22,544 connection records.

Each row represents a single network connection described by 41 features and a class label. The features fall into four families:

- **Basic features (1–9):** duration, protocol type (TCP/UDP/ICMP), service (HTTP, FTP, SMTP, ...), source and destination byte counts, and connection flags.
- **Content features (10–22):** payload-related signals such as failed login attempts, root shell access, and file creation operations.
- **Time-based traffic features (23–31):** statistics computed over a two-second window, e.g. how many connections to the same host occurred in that time and how many had SYN errors.
- **Host-based traffic features (32–41):** the same kind of statistics computed over the past 100 connections to the same host.

Three of the 41 features are categorical (`protocol_type`, `service`, `flag`); the rest are numerical.

2.3 Class Distribution and the Imbalance Problem

The 22 specific attack labels in the raw dataset are conventionally collapsed into four high-level attack families plus Normal. Table 1 gives the resulting class distribution and Figure 1 plots it on a log scale.

Table 1: NSL-KDD class distribution after the standard 5-class mapping.

Class	Train	%	Test	%
Normal	67,343	53.46	9,711	43.08
DoS	45,927	36.46	7,458	33.08
Probe	11,656	9.25	2,421	10.74
R2L	995	0.79	2,887	12.81
U2R	52	0.04	67	0.30
Total	125,973	100	22,544	100

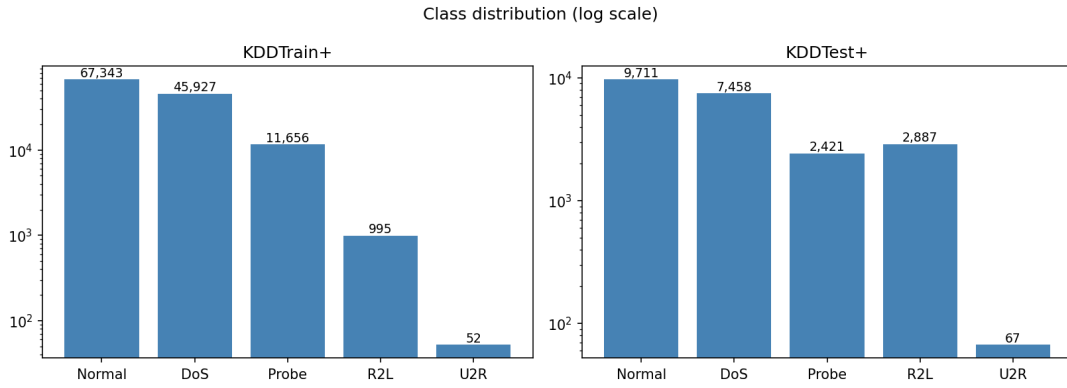


Figure 1: Class distribution on a log scale. There are roughly three orders of magnitude separating Normal/DoS from U2R.

Two things stand out.

The training set is severely imbalanced. Normal and DoS together account for roughly 90% of the rows. U2R has only 52 training samples — 0.04% of the data. A model that simply predicts “Normal” for every connection would already score 53% accuracy; this is why accuracy alone is a misleading metric on this problem.

The test set has more rare attacks than the training set relative to its size. R2L is 0.79% of training but 12.81% of testing. This is not a mistake. NSL-KDD’s test set deliberately includes attack subtypes that are absent or under-represented in training, so that test performance reflects how the model handles *novel* attacks rather than just memorised ones. This design is what makes NSL-KDD harder than its predecessor and is responsible for the cross-validation-vs-test-score gap discussed in Section 5.

2.4 Attack Categories Explained

- **Denial of Service (DoS).** The attacker floods a service with so much traffic that legitimate users can’t reach it. Examples in NSL-KDD: `neptune` (SYN flood), `smurf`, `teardrop`.
- **Probe.** Reconnaissance — the attacker scans the network looking for weak spots. Examples: `nmap`, `portsweep`, `satan`.

- **Remote-to-Local (R2L).** The attacker is on an external machine and tries to gain access to an internal one, e.g. via password guessing or exploits. Examples: `guess_passwd`, `ftp_write`, `warezclient`.
- **User-to-Root (U2R).** The attacker already has a normal user account and tries to escalate to root. Examples: `buffer_overflow`, `rootkit`, `loadmodule`.

R2L and U2R are both rare in the dataset and the most dangerous in practice, because they imply that the attacker has either gotten in (R2L) or taken control (U2R). They are also the hardest to detect from connection-level features, as the experiments will show.

3 Methodology

3.1 High-Level Pipeline

The pipeline has the following stages:

Load → Encode & Scale → Split-aware SMOTE → Train → Evaluate → Save

Each stage lives in its own Python module so that swapping out a single piece (a different algorithm, a different dataset) does not break the rest. Section 4 walks through the code; this section describes *why* each step is the way it is.

3.2 Preprocessing

3.2.1 Encoding categorical features

The three categorical columns must be converted to numbers before any classifier can use them. We use one-hot encoding, which creates one new binary column per category. After encoding the dimensionality grows from 41 to 122, with most of the growth coming from the `service` column (about 70 distinct values like `http`, `ftp_data`, `telnet`, ...).

A subtle point: we encode train and test *together* so that any service strings that appear only in the test set still have a column. If we encoded each split independently, the test set would have different columns from the training set and the model could not be applied.

3.2.2 Standardisation

After encoding, all numerical features are standardised to zero mean and unit variance, using a scaler fit *only* on the training data. Standardisation matters most for Logistic Regression, which interprets coefficient sizes directly; tree-based methods do not require it but are not harmed by it. Fitting the scaler on training data only is a basic discipline rule to avoid leaking test-set distribution information into the model.

3.3 Class Imbalance: SMOTE

The Synthetic Minority Over-sampling Technique (SMOTE) [3] addresses class imbalance by generating synthetic minority-class samples. For a given minority sample, SMOTE picks one of its k nearest neighbours from the same class and creates a new sample by interpolating between the two. Compared with simple random oversampling, this produces *novel* synthetic samples rather than duplicates and reduces the risk of overfitting to a handful of minority points.

The non-obvious part of SMOTE is *when* to apply it. We apply SMOTE *inside each cross-validation fold*, on the training side only. Resampling the entire training set before splitting into CV folds would generate synthetic minority samples that are interpolations between the training and validation neighbours, leaking validation information back into the training set.

The result is artificially inflated cross-validation scores. After cross-validation we apply SMOTE one more time on the full training set for the final model fit; the held-out KDDTest+ split is never touched by SMOTE.

3.4 Algorithms

Each algorithm is configured with deliberately conservative hyperparameters. Heavy tuning would cloud the comparison because each algorithm has different sensitivity to its hyperparameter space.

3.4.1 Logistic Regression

A linear classifier that models the log-odds of class membership as a weighted sum of features. We use the multinomial formulation (one set of weights per class) with the L-BFGS solver and `max_iter=1000`.

3.4.2 Random Forest

An ensemble of decision trees [4] trained on bootstrap samples with random feature subsetting at each split. We use 100 trees with no maximum depth and `class_weight='balanced'` so the trees give extra weight to minority-class samples at each split. Random Forest is robust to noisy features, handles mixed feature types, and produces a feature importance score for free.

3.4.3 XGBoost

A gradient-boosting ensemble [5] that fits trees sequentially, where each new tree is trained to correct the residual errors of the previous ensemble. We use 200 trees of maximum depth 6, learning rate 0.1, and the histogram-based tree construction algorithm for speed. XGBoost adds L1/L2 regularisation on the leaf weights and has native multi-class support via the `multi:softprob` objective.

3.5 Evaluation

3.5.1 Cross-validation and held-out test

We use stratified 5-fold cross-validation on the training set followed by a final evaluation on KDDTest+. Stratified folds preserve the original class proportions in each fold. “5-fold” (rather than the proposal’s 10-fold) was chosen because at $n = 125,973$ rows the variance of the CV estimate is already small at 5 folds, and 5-fold cuts the runtime roughly in half. We report mean and standard deviation across folds.

3.5.2 Metrics

We report the standard four metrics, both per-class and as weighted/macro averages:

$$\begin{aligned}\text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \\ \text{Accuracy} &= \frac{TP + TN}{\text{Total}}.\end{aligned}$$

We additionally compute one-versus-rest ROC-AUC. The most informative single number on imbalanced data is *macro F1*, which averages per-class F1 scores equally; weighted F1 weights by class support and tends to reward majority-class accuracy.

4 Implementation

4.1 Project Layout

```
machine_learning/
  src/
    config.py           # Paths, label maps, hyperparameters
    data_loader.py      # Load NSL-KDD, map 22 attacks to 5 classes
    preprocess.py       # One-hot encode + scale + label encode
    models.py          # Define LR, RF, XGBoost
    evaluate.py         # Metrics, confusion matrix, plots
    run_experiments.py  # Entry point
  dataset/
    archive/nsl-kdd/    # KDDTrain+.txt, KDDTest+.txt
  results/              # Per-model JSON + comparison.csv
  figures/              # Class distribution, CMs, FI plots
  requirements.txt
  .venv/                # Virtual environment
```

The split into modules follows the principle that each file should have one job. Swapping NSL-KDD for CICIDS2017 in the future would only require changes to `data_loader.py`; nothing else needs to know.

4.2 Loading and Label Mapping

The 22 specific attack labels in the raw NSL-KDD files are mapped to the 5-class taxonomy via a dictionary in `config.py`. The loader drops the difficulty score (we do not use difficulty stratification in this study) and applies the mapping.

```
1 def _load_split(path):
2     df = pd.read_csv(path, header=None, names=NSL_KDD_COLUMNS)
3     df = df.drop(columns=["difficulty"])
4     df["label"] = df["label"].map(
5         lambda x: ATTACK_TO_CATEGORY.get(x, "R2L")
6     )
7     return df
8
9 def load_nsl_kdd():
10     train_df = _load_split(TRAIN_FILE)
11     test_df = _load_split(TEST_FILE)
12     return train_df, test_df
```

Listing 1: Loading and 5-class mapping (`data_loader.py`).

4.3 Preprocessing

```
1 def encode_features(train_df, test_df):
2     X_train, y_train = _split_xy(train_df)
3     X_test, y_test = _split_xy(test_df)
4     combined = pd.concat([X_train, X_test], keys=["train", "test"])
5     combined_encoded = pd.get_dummies(
6         combined,
7         columns=CATEGORICAL_FEATURES,
8         drop_first=False,
```

```

9         ).astype(np.float32)
10        X_train_enc = combined_encoded.loc["train"].reset_index(drop=True)
11        X_test_enc  = combined_encoded.loc["test"].reset_index(drop=True)
12        return X_train_enc, y_train, X_test_enc, y_test

```

Listing 2: One-hot encoding over the union of train/test categories (preprocess.py).

The reason we encode over the concatenation rather than fitting an encoder on train alone is documented in the module's docstring: rare `service` strings appear only in the test set and must still be representable.

4.4 Per-Fold SMOTE

The non-trivial part of the experiment loop is making sure SMOTE only resamples the training side of each CV fold.

```

1 def cv_with_smote(model, X, y, cv_folds=CV_FOLDS):
2     skf = StratifiedKFold(n_splits=cv_folds, shuffle=True,
3                           random_state=RANDOM_SEED)
4     fold_scores = []
5     for fold_idx, (tr_idx, va_idx) in enumerate(skf.split(X, y), 1):
6         X_tr, X_va = X[tr_idx], X[va_idx]
7         y_tr, y_va = y[tr_idx], y[va_idx]
8
9         # SMOTE k_neighbors must be <= (smallest class count - 1).
10        min_count = pd.Series(y_tr).value_counts().min()
11        k = max(1, min(5, min_count - 1))
12        sm = SMOTE(random_state=RANDOM_SEED, k_neighbors=k)
13        X_tr_res, y_tr_res = sm.fit_resample(X_tr, y_tr)
14
15        m = clone(model)
16        m.fit(X_tr_res, y_tr_res)
17        y_pred = m.predict(X_va)
18        fold_scores.append(
19            f1_score(y_va, y_pred, average="weighted",
20                    zero_division=0)
21        )
22    return np.mean(fold_scores), np.std(fold_scores)

```

Listing 3: Stratified k-fold CV with SMOTE inside each fold (run_experiments.py).

The defensive clamp on `k_neighbors` matters because if a fold happens to have very few U2R samples, SMOTE's default `k=5` is too high and the algorithm crashes.

4.5 Models

```

1 def build_models():
2     return {
3         "Logistic Regression": LogisticRegression(
4             max_iter=1000,
5             solver="lbfgs",
6             random_state=RANDOM_SEED,
7         ),
8         "Random Forest": RandomForestClassifier(
9             n_estimators=100,
10            max_depth=None,
11            n_jobs=-1,
12            random_state=RANDOM_SEED,
13            class_weight="balanced",
14        ),
15         "XGBoost": XGBClassifier(
16             n_estimators=200,

```

```

17         max_depth=6,
18         learning_rate=0.1,
19         objective="multi:softprob",
20         num_class=5,
21         tree_method="hist",
22         n_jobs=-1,
23         random_state=RANDOM_SEED,
24         eval_metric="mlogloss",
25     ),
26 }

```

Listing 4: Three classifiers under comparison (models.py).

4.6 Evaluation Utilities

The evaluation module computes overall metrics, per-class tables, and saves confusion matrices and feature importance plots to the `figures/` directory. Per-class precision, recall, and F1 are computed via `classification_report` from scikit-learn. ROC-AUC is computed under the one-versus-rest binarisation.

4.7 Reproducibility

Every random source is seeded (`RANDOM_SEED=42`). The full experiment runs with:

```

python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
PYTHONPATH=src .venv/bin/python src/run_experiments.py

```

4.8 Source Code Repository

All code, LaTeX sources for this report and the accompanying research paper, generated figures, and per-model result files are published in a public GitHub repository:

<https://github.com/SudipAdh/NETWORK-INTRUSION-DETECTION>

The repository mirrors the layout described in this report (`src/`, `figures/`, `results/`, `assignment_report/`, `research_paper/`). The NSL-KDD dataset itself is not redistributed and must be downloaded separately — the README documents the canonical Kaggle and University of New Brunswick mirrors.

5 Results and Discussion

5.1 Overall Comparison

Table 2 summarises the held-out test results.

Table 2: Overall performance on KDDTest+. CV F1 is the mean weighted F1 across 5 folds on the training set.

Algorithm	Acc. (%)	Wt. F1	Macro F1	ROC-AUC	CV F1	Time (s)
Logistic Regression	77.91	0.7631	0.5745	0.8970	0.9756	330.3
Random Forest	75.05	0.7112	0.5382	0.9476	0.9987	53.8
XGBoost	78.66	0.7582	0.6337	0.9537	0.9991	90.1

XGBoost wins on three of the four held-out metrics (accuracy, macro F1, ROC-AUC). Random Forest is the fastest to train. Logistic Regression edges out the other two on weighted

F1 by less than half a percentage point but, as the per-class breakdown will show, this is largely because the weighted average gives extra credit for getting Normal and DoS right.

5.2 The Cross-Validation versus Held-Out Gap

The most striking number in Table 2 is not in the comparison columns at all. The *cross-validation* weighted F1 scores are 0.9756, 0.9987, and 0.9991 — effectively perfect for the two ensemble methods. The held-out test scores are 0.7631, 0.7112, and 0.7582. That is a gap of roughly 22 percentage points.

This gap is not the usual “the model overfit to the training data” overfitting. The training-versus-CV gap is small. The gap is between the *training distribution* (from which both train and CV folds are drawn) and the *test distribution*, which contains attack subtypes that are absent or rare in training. This is by design — it is what makes NSL-KDD a hard benchmark.

The practical lesson is that papers that report only cross-validation results on NSL-KDD are not measuring what they appear to be measuring. They are measuring how well the model fits the training distribution, not how well it would generalise to a slightly different attack mix. This study reports both numbers so the gap is visible.

5.3 Per-Class Performance

Table 3 presents the per-class F1 scores on KDDTest+.

Table 3: Per-class F1-scores on KDDTest+.

Algorithm	Normal	DoS	Probe	R2L	U2R
Logistic Regression	0.8058	0.9028	0.7344	0.2976	0.1320
Random Forest	0.7800	0.8447	0.7614	0.1048	0.2000
XGBoost	0.8095	0.8788	0.7848	0.2597	0.4356

A few patterns emerge.

All models do well on Normal and DoS. F1 ranges from 0.78 to 0.90, which means these classes are largely a solved problem for any of the three models.

All models struggle on R2L. The best performer on this class is actually Logistic Regression (0.2976), with XGBoost in second place (0.2597). Random Forest collapses on R2L (F1 = 0.1048). The reason LR does best here is that with class-balanced training and simple linear boundaries, LR produces relatively diffuse decisions that occasionally help on noisy minority classes. The trees, particularly the bagging ensemble, are more confident in their majority-class predictions and miss more R2L cases.

XGBoost is the only model with usable U2R detection. Its U2R F1 of 0.4356 is more than twice that of Random Forest (0.2000) and more than triple that of LR (0.1320). The gradient boosting mechanism — where each new tree is trained to correct the residuals of the previous ensemble — appears to genuinely help here, because the later trees specialise in the U2R cases that earlier trees miss.

5.4 Confusion Matrices

Figures 2, 3, and 4 show the confusion matrices for each model on KDDTest+. The right-hand pane is row-normalised so the diagonal is per-class recall.

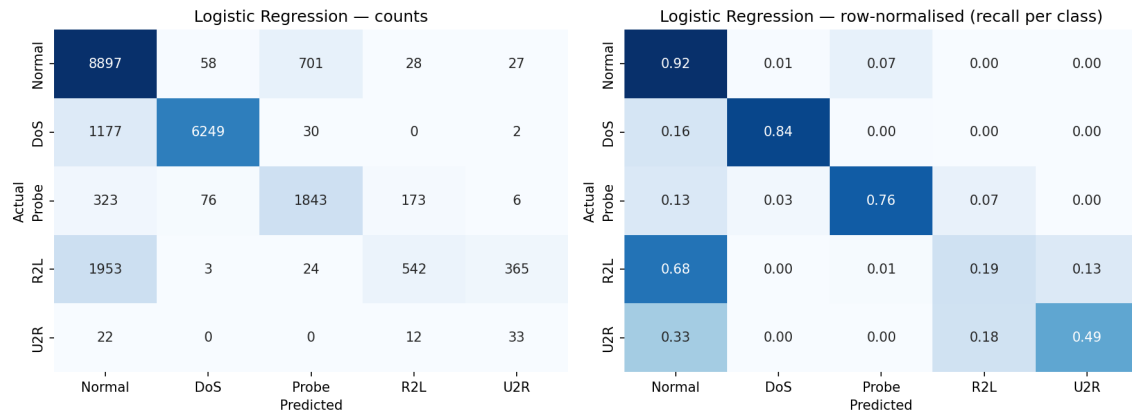


Figure 2: Logistic Regression confusion matrix on KDDTest+.

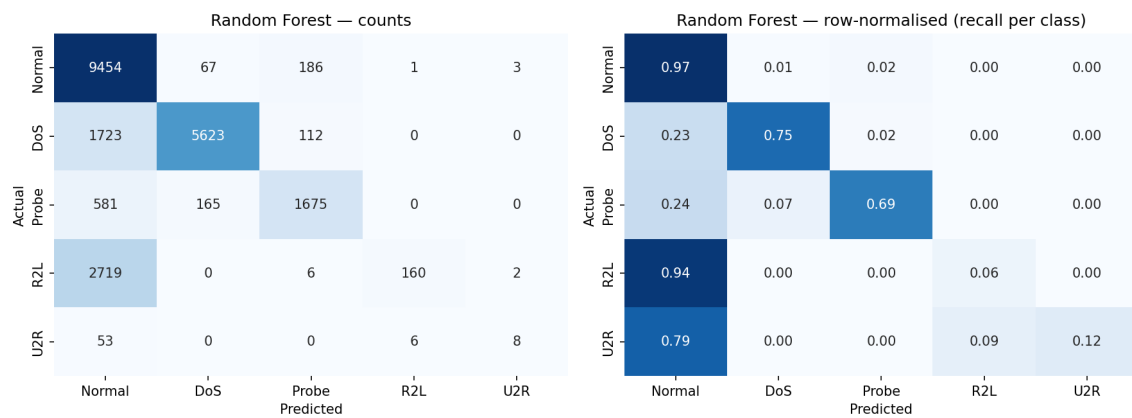


Figure 3: Random Forest confusion matrix on KDDTest+.

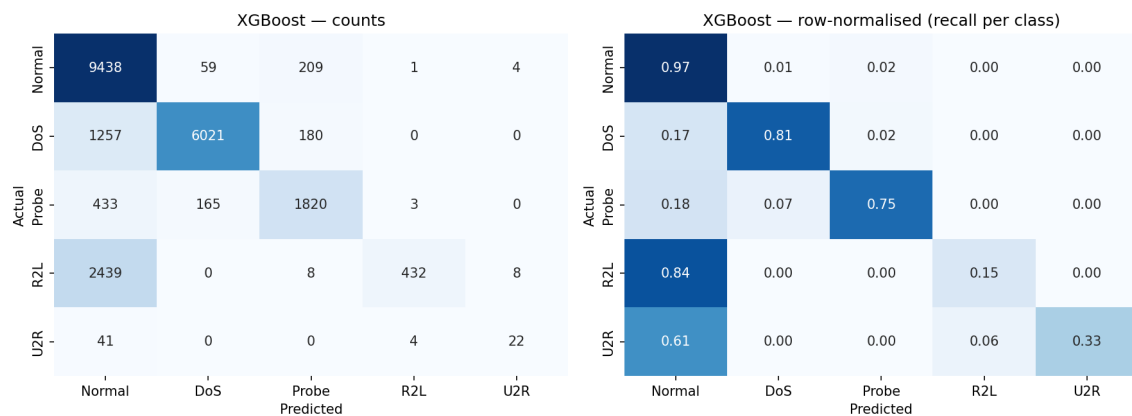


Figure 4: XGBoost confusion matrix on KDDTest+.

The confusion matrices give the security implication directly. Looking at the XGBoost matrix in Figure 4, 84% of R2L test attacks are misclassified as Normal (2,439 out of 2,887). For U2R, 61% are misclassified as Normal (41 out of 67). These are the most dangerous attack categories in the dataset — unauthorised remote access and privilege escalation — and the model’s most common mistake is to label them as benign traffic.

The takeaway is sobering: a deployment that relies solely on a classical-ML detector trained on NSL-KDD-style features would miss the majority of the very attacks that matter most.

In production, this kind of classifier needs to be paired with anomaly detection on payloads, behavioural baselines on user accounts, or human review.

5.5 Feature Importance

Random Forest and XGBoost both produce feature importance scores. For Logistic Regression we use the mean absolute coefficient across classes as a rough analogue. Figure 5 shows the top 15 features as ranked by XGBoost.

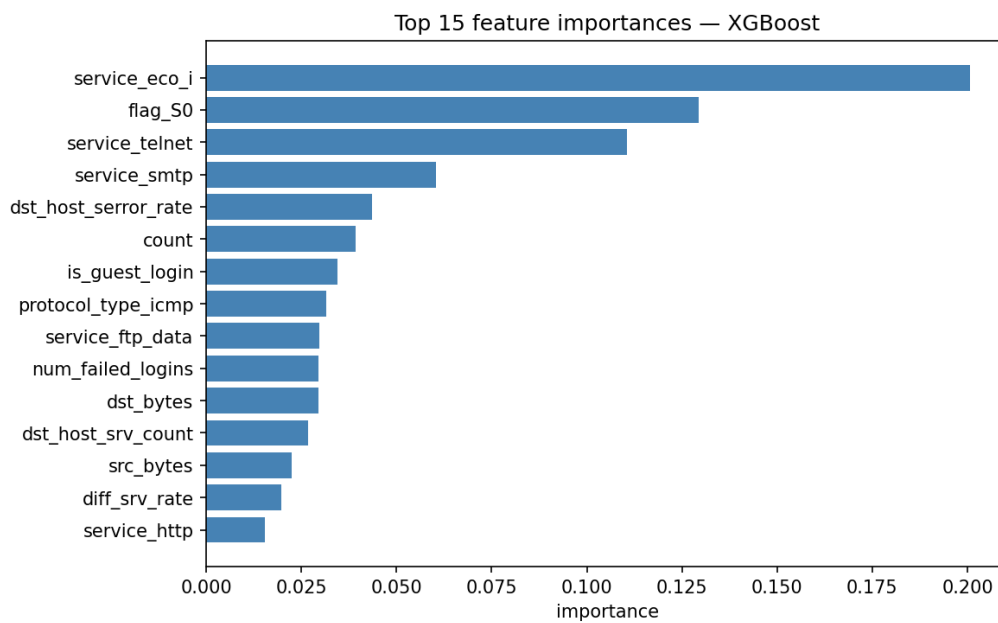


Figure 5: Top 15 feature importances from XGBoost.

The top features are interpretable.

- **service_eco_i** (~20% of total importance) marks ICMP echo traffic, which appears heavily in probe attacks.
- **flag_S0** (~13%) marks connections where a SYN was sent but no SYN-ACK returned — the canonical signature of SYN floods and port scans.
- **service_telnet** (~11%) flags Telnet activity, which is rare in legitimate modern environments and shows up in R2L brute-force attempts.
- **service_smtp** (~6%) flags mail traffic, which figures in some R2L and DoS variants.
- **dst_host_serror_rate** (~5%) is the rate of connection errors to a target host — another classic scanning signature.

The first three features alone account for roughly 40% of the model’s split decisions. This is a useful practical result: an analyst running a small monitoring deployment could focus on these few quantities and capture most of what the trained model would. The trained model adds value by quantifying their interaction with thirty-eight other signals, but the headline features are exactly what a security textbook would tell you to watch.

5.6 Wall-Clock Time

Random Forest is the fastest of the three (53.8 s end-to-end including 5-fold CV), XGBoost is intermediate (90.1 s), and Logistic Regression is, surprisingly, the slowest (330.3 s). The reason LR is slow is that after SMOTE the training set grows from 125,973 to 336,715 rows, and on 122 features the L-BFGS optimiser requires many passes over this larger matrix to converge to a strict tolerance. Tree-based methods, by contrast, scan each feature once per split and parallelise easily across cores. This is a useful counterintuition — “simple” linear models are not always faster than “complex” ensembles.

5.7 Summary of Findings

1. **XGBoost is the best overall model on this benchmark**, winning on accuracy, macro F1, ROC-AUC, and rare-class detection.
2. **All three models are strong on Normal and DoS** ($F1 > 0.78$) but weak on R2L ($F1 \leq 0.30$).
3. **The 22-percentage-point CV-vs-test gap** is the single largest effect in the experiment and matters more than the differences between models.
4. **Macro F1 is the right headline metric** on imbalanced data; weighted F1 and accuracy hide the rare-class failures.
5. **A few features carry most of the signal** (`service_eco_i`, `flag_S0`, `service_telnet`). This is reassuring for interpretability.

6 Limitations and Future Work

This study has clear limitations.

Single dataset. All conclusions are drawn from NSL-KDD. The underlying network traffic is from 1998 and lacks modern attack patterns: HTTP-based exploits, encrypted-channel exfiltration, modern DDoS, web shells, modern botnets. A natural follow-up is to retrain the same pipeline on CICIDS2017 [10], a 2017 benchmark with ~ 2.8 million labelled flows and 14 attack categories including Heartbleed, Botnet, and Web Attacks. We have left a placeholder `data_loader` for this work.

Classical algorithms only. We do not compare against deep learning. Existing literature suggests that on NSL-KDD, well-tuned classical models are competitive with deep models in accuracy while being orders of magnitude faster to train. On larger, modern benchmarks like CICIDS2017 the picture changes; deep learning would be a sensible next step.

Conservative hyperparameters. We did not run a hyperparameter search. The trade-off is that aggressive tuning of one model would distort the comparison. With more compute, a Bayesian or grid search per model with the same budget for each would tighten the numbers but probably not change the qualitative ranking.

Single resampling strategy. We only used SMOTE. ADASYN, Borderline-SMOTE, SMOTE-Tomek, and undersampling techniques would each give slightly different rare-class behaviour. Cost-sensitive learning — adjusting the loss so that rare-class errors are penalised more — is a particularly interesting direction we did not explore.

R2L detection remains poor. The strongest model in the study still recalls only 15% of R2L attacks on the test set. This is a fundamental limit of connection-level features for R2L-style attacks, where the malicious behaviour lives in the payload (e.g. password attempts, exploit strings) rather than in connection statistics. A production system would augment this kind of model with payload inspection or behavioural baselining.

7 Conclusion

This project compared three classical machine learning algorithms — Logistic Regression, Random Forest, and XGBoost — on the full five-class NSL-KDD intrusion detection problem under identical preprocessing and evaluation conditions. The pipeline was reproducible end-to-end: 5-fold cross-validation with SMOTE applied per fold, a held-out evaluation on the official test split, and per-class metrics throughout. XGBoost emerged as the strongest model on the held-out test set (78.66% accuracy, 0.6337 macro F1, 0.9537 ROC-AUC) and was the only model with credible U2R detection ($F1 = 0.4356$). Logistic Regression was unexpectedly strong on R2L by virtue of class-balanced training; Random Forest was fastest to train but weakest on rare classes.

Two findings deserve more emphasis than they are usually given in NSL-KDD studies. The first is the 22-percentage-point gap between cross-validation and held-out test scores — a property of NSL-KDD’s deliberate inclusion of unseen attack subtypes in the test set, and a number that papers reporting only CV results are quietly hiding. The second is that all three models, even the best, miss the large majority of R2L attacks. Classical-ML detectors trained on connection-level features should not be deployed as the sole line of defence against R2L-style threats.

What I learned from doing the project is more practical than the numbers themselves. The amount of work that goes into a fair comparison — making sure preprocessing is identical across models, that SMOTE is applied per fold rather than once at the start, that the test set is never touched until the very end — is much greater than the work of running each algorithm in isolation. Many of the published NSL-KDD studies skip one or more of these steps and still report “state-of-the-art” numbers; once you do the careful version, the numbers are lower but defensible. For the same effort that produces a single mediocre paper, a careful pipeline produces something that can be extended to other datasets, other algorithms, and other evaluation settings without rewriting from scratch.

References

References

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pp. 1–6, 2009.
- [2] KDD Cup 1999 Data, UCI Machine Learning Repository. <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [6] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, “Network intrusion detection system: A systematic study of machine learning and deep learning approaches,” *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, e4150, 2021.

- [7] L. Dhanabal and S. P. Shantharajah, “A study on NSL-KDD dataset for intrusion detection system based on classification algorithms,” *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 4, no. 6, pp. 446–452, 2015.
- [8] B. Ingre and A. Yadav, “Performance analysis of NSL-KDD dataset using ANN,” *Proc. International Conference on Signal Processing and Communication Engineering Systems (SPACES)*, pp. 92–96, 2015.
- [9] J. Gu and S. Lu, “An effective intrusion detection approach using SVM with naïve Bayes feature embedding,” *Computers & Security*, vol. 103, p. 102158, 2021.
- [10] R. Panigrahi and S. Borah, “A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems,” *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.
- [11] “Anomaly-based intrusion detection on benchmark datasets for network security: a comprehensive evaluation,” *Scientific Reports*, vol. 16, 2026.
- [12] “Advanced IDS: a comparative study of datasets and machine learning algorithms for network flow-based intrusion detection systems,” *Applied Intelligence*, 2025.
- [13] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix A: Environment and Reproduction

Python: 3.13.2

Required libraries (requirements.txt):

```
pandas
numpy
scikit-learn
matplotlib
seaborn
xgboost
imbalanced-learn
joblib
```

macOS dependency: XGBoost requires OpenMP. Install with `brew install libomp`.

Reproducing the experiment:

```
# 1. Create the virtual environment
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt

# 2. Run the full pipeline
PYTHONPATH=src .venv/bin/python src/run_experiments.py
```

The script writes to **results/** (one JSON and one CSV per model, plus the master **comparison.csv**) and to **figures/** (class distribution, three confusion matrices, three feature importance plots). All randomness is seeded with **RANDOM_SEED=42**.

Source code: The complete project — code, reports, figures, and per-model results — is hosted at <https://github.com/SudipAdh/NETWORK-INTRUSION-DETECTION>. Clone the repository, place the NSL-KDD train/test files under **dataset/archive/** (download links in the README), and run the command above to reproduce every figure and table in this report.

Appendix B: Selected Code Listings

B.1: Configuration (config.py)

```

1 RANDOM_SEED = 42
2 CV_FOLDS = 5
3 CATEGORICAL_FEATURES = ["protocol_type", "service", "flag"]
4 CLASS_LABELS = ["Normal", "DoS", "Probe", "R2L", "U2R"]
5 ATTACK_TO_CATEGORY = {
6     "normal": "Normal",
7     "back": "DoS", "neptune": "DoS", "smurf": "DoS", ...,
8     "ipsweep": "Probe", "nmap": "Probe", "satan": "Probe", ...,
9     "ftp_write": "R2L", "guess_passwd": "R2L", ...,
10    "buffer_overflow": "U2R", "rootkit": "U2R", ...,
11 }

```

Listing 5: Central configuration: paths, label maps, hyperparameters.

B.2: End-to-End Driver (run_experiments.py)

```

1 def main():
2     train_df, test_df = load_nsl_kdd()
3     plot_class_distribution(train_df, test_df, ...)
4
5     X_train, y_train, X_test, y_test, feature_names, _ = (
6         full_pipeline(train_df, test_df)
7     )
8
9     sm = SMOTE(random_state=RANDOM_SEED, k_neighbors=5)
10    X_train_smote, y_train_smote = sm.fit_resample(X_train, y_train)
11
12    for name, model in build_models().items():
13        cv_mean, cv_std = cv_with_smote(model, X_train, y_train)
14        model.fit(X_train_smote, y_train_smote)
15        y_pred = model.predict(X_test)
16        y_proba = model.predict_proba(X_test)
17        overall = overall_metrics(y_test, y_pred, y_proba)
18        per_class = per_class_table(y_test, y_pred)
19        save_results(name, overall, per_class, predictions=y_pred)
20        plot_confusion_matrix(y_test, y_pred, name, ...)
21        plot_feature_importance(feature_names, ..., name, ...)

```

Listing 6: Main pipeline: load -> preprocess -> CV+SMOTE -> final fit -> test eval.

Appendix C: Full Per-Class Tables

Table 4: Logistic Regression — per-class results on KDDTest+.

	precision	recall	f1-score	support
Normal	0.7191	0.9162	0.8058	9,711
DoS	0.9785	0.8379	0.9028	7,458
Probe	0.7094	0.7613	0.7344	2,421
R2L	0.7179	0.1877	0.2976	2,887
U2R	0.0762	0.4925	0.1320	67

Table 5: Random Forest — per-class results on KDDTest+.

	precision	recall	f1-score	support
Normal	0.6507	0.9735	0.7800	9,711
DoS	0.9604	0.7540	0.8447	7,458
Probe	0.8464	0.6919	0.7614	2,421
R2L	0.9581	0.0554	0.1048	2,887
U2R	0.6154	0.1194	0.2000	67

Table 6: XGBoost — per-class results on KDDTest+.

	precision	recall	f1-score	support
Normal	0.6936	0.9719	0.8095	9,711
DoS	0.9641	0.8073	0.8788	7,458
Probe	0.8209	0.7518	0.7848	2,421
R2L	0.9818	0.1496	0.2597	2,887
U2R	0.6471	0.3284	0.4356	67